

System design - Writer & Reader nodes

Here is a detailed breakdown of the technical terms and concepts.

1. Writer Node (Primary Instance)

The **Writer Node** is the central hub of your database cluster. It is the only node that has "Write" permissions, meaning it can change the state of the data.

- **How it works:** When an application sends an **INSERT**, **UPDATE**, or **DELETE** command, it must go to this node. It ensures **ACID compliance** (Atomicity, Consistency, Isolation, Durability) by being the single authority on what the data currently looks like.
 - **Example 1 (E-commerce):** When you click "Place Order," the database must subtract "1" from the inventory count. This happens on the **Writer Node** to prevent two people from buying the last item at the exact same millisecond.
 - **Example 2 (Social Media):** When you edit your profile bio, that "Update" query is handled by the **Writer Node** to ensure your new bio is saved permanently.
 - **Official Resource:** [AWS Aurora DB Clusters & Instances](#)
-

2. Reader Nodes (Read Replicas)

Reader Nodes are "read-only" copies of your primary database. They stay synchronized with the Writer Node and serve data to users without putting any load on the main engine.

- **How it works:** They use **Asynchronous Replication**. The Writer sends a log of its changes to the Readers, which update themselves. This allows you to **Scale Horizontally**—if you have too much traffic, you just add more Readers.
 - **Example 1 (News Website):** 10,000 people are reading the same breaking news article. Each of those "Read" requests is sent to a pool of **Reader Nodes**, keeping the Writer Node free to handle journalists posting new stories.
 - **Example 2 (Banking App):** When you open your app to check your "Transaction History," you are likely reading from a **Reader Node**.
 - **Official Resource:** [Google Cloud - Working with Read Replicas](#)
-

3. Replication Lag

Replication Lag is the small amount of time (usually milliseconds) it takes for a change made on the Writer Node to show up on the Reader Nodes.

- **Why it happens:** Since data has to travel over a network from the Writer to the Reader, there is a physical delay. If the network is busy or the database is under heavy load, this lag can increase.
 - **Example 1 (Password Reset):** You change your password (Writer). You immediately try to log in, but the login service checks a Reader Node that hasn't received the update yet. You get an "Incorrect Password" error for a split second.
 - **Example 2 (Follow Count):** You follow a creator on Instagram. The "Follow" is recorded on the Writer, but when you refresh the page immediately, the count might not have changed yet because the Reader is still 200ms behind.
 - **Official Resource:** [Monitoring Read Replication Lag \(AWS\)](#)
-

4. Read-to-Write Ratio

This is a metric used to decide how to design your system. It compares how many queries are **fetching data** (Reads) versus how many are **saving data** (Writes).

- **High Read Ratio (e.g., 99:1):** Systems like **Twitter** or **Wikipedia**. Millions read content, but very few actually post or edit. This is where Reader Nodes are most valuable.
 - **High Write Ratio (e.g., 1:1 or 1:5):** Systems like **IoT Sensor Logging** or **Stock Market Tickers**. Data is constantly flowing in. Here, adding more Readers won't help; you might need "Sharding" instead.
 - **Official Resource:** [Microsoft Azure - Scaling Reads with Replicas](#)
-

5. Horizontal vs. Vertical Scaling

This explains *how* you handle more traffic.

- **Vertical Scaling:** Buying a "bigger" server with more RAM and CPU. This is what you do for your **Writer Node** because you can usually only have one. It has a physical limit (you can't buy a server infinitely large).
- **Horizontal Scaling:** Adding "more" servers. This is what you do with **Reader Nodes**. You can have 5, 10, or even 15 replicas (in AWS Aurora) working together.
- **Official Resource:** [AWS - Scaling your RDS Instance](#)